Step-by-Step Implementation Plan: Adaptive Unlearning for Dynamic Fairness

Research Methodology

1 Problem Restatement

The goal is to design an adaptive machine unlearning framework that continuously monitors a deployed model's fairness metrics over time, detects bias re-emergence due to data distribution shifts or biased injections, and triggers selective unlearning of problematic data points without full retraining. The framework must balance fairness, utility, and computational efficiency.

2 Overall Architecture

The proposed solution consists of five core modules:

1. **Online Fairness Monitor**- tracks group fairness metrics on a sliding window.
2. **Bias Detection Trigger**- flags fairness violations with statistical tests.
3. **Bias Source Identification**- identifies data points or features causing bias (using influence functions or Shapley values).
4. **Selective Unlearning Engine**- removes influence of identified points via incremental unlearning.
5. **Utility Preservation & Audit Log**- maintains model performance and records unlearning actions.

3 Step-by-Step Implementation Methodology

3.1 Step 1: Data Stream Simulation and Preprocessing

- **Synthetic data**: Generate a streaming dataset with controllable concept drift and bias injection (e.g., changing correlation between a protected attribute and the label).

- **Real-world data**: Use online lending (e.g., Home Credit Default) or dynamic healthcare records (e.g., MIMIC-IV). Split into time-ordered chunks.
- **Protected attributes**: Define sensitive features (e.g., gender, race) and fairness metrics: statistical parity difference (SPD) and equalized odds difference (EOD).

3.2 Step 2: Base Model Deployment

- Train an initial model (e.g., logistic regression or a small neural network) on the first data batch.
- Store model parameters θ_0 and maintain a reservoir of training data indices with their influence scores (initially all).

3.3 Step 3: Online Fairness Monitoring Module

- For each incoming mini-batch B_t (size m):
  1. Compute predictions $\hat{y}$ using current model θ_{t-1}.
  2. Calculate group fairness metrics on a sliding window W of the last L predictions and true labels.
  3. Maintain running estimates of $SPD = |P(\hat{y} = 1|A = 0) - P(\hat{y} = 1|A = 1)|$ and $EOD = \frac{1}{2}(|TPR_0 - TPR_1| + |FPR_0 - FPR_1|)$.

3.4 Step 4: Bias Detection Trigger

- Use a cumulative sum control chart (CUSUM) or a sliding-window Kolmogorov–Smirnov test to detect when SPD or EOD exceeds a threshold τ (e.g., 0.1) for k consecutive windows.
- Upon detection, raise a `fairness_violation` flag and proceed to source identification.

3.5 Step 5: Bias Source Identification via Influence Functions

- Goal: Find which training points (or recent points) contribute most to the current bias.
- Approximate influence of removing a point z on fairness metric $\mathcal{F}$ using influence functions:

$$\mathcal{I}_{\text{up,loss}}(z, z_{\text{test}}) = -\nabla_\theta \mathcal{L}(z_{\text{test}}, \theta)^\top H_\theta^{-1} \nabla_\theta \mathcal{L}(z, \theta)$$

  where H_θ is the Hessian of the training loss.

- For fairness, compute influence on a fairness loss $\mathcal{L}_{\text{fair}}$ (e.g., difference in true positive rates). Points with highest positive influence on unfairness are candidates for unlearning.
- To reduce computation, use a stochastic approximation (e.g., LiSSA) and evaluate only on a random subset of training points or recent points in a sliding window.

3.6 Step 6: Selective Unlearning Engine

- Given a set $\mathcal{U}$ of identified problematic points (size $n_u \ll N$), apply incremental unlearning without full retraining.
- **Method**: Use a Newton-step update based on influence functions (approximate unlearning):

$$\theta_{\text{new}} = \theta_{\text{old}} + \frac{1}{N} H_\theta^{-1} \sum_{z \in \mathcal{U}} \nabla_\theta \mathcal{L}(z, \theta_{\text{old}})$$

- If the above causes utility drop, apply a hybrid approach: (a) store a small memory of correctly classified and fairly behaving points; (b) perform a few gradient steps on that memory after unlearning (replay).
- For large $\mathcal{U}$, fall back to *partitioned unlearning*: split model into shards (e.g., using SISA – Sharded, Isolated, Sliced, Aggregated) and retrain only the affected shard.

3.7 Step 7: Utility Preservation Constraints

- After unlearning, evaluate predictive accuracy on a validation window.
- If accuracy drops more than δ (e.g., 5%), apply a fine-tuning step with a regularization term that penalizes deviation from the previous model's predictions on a set of anchor points (e.g., using elastic weight consolidation).
- Alternatively, maintain a small buffer of recent, fairly-behaved data and interleave its gradients during unlearning (experience replay).

3.8 Step 8: Audit Trail and Adaptive Control

- Log each unlearning event: timestamp, number of removed points, influence scores, fairness metrics before/after, computational cost.
- Implement a feedback loop: if fairness violations recur frequently, adjust the bias detection threshold τ or the influence estimation window size.
- Provide an explainability report: top-k features or data points causing bias.

3.9 Step 9: Evaluation Protocol

Compare against three baselines:

1. **Periodic full retraining** (e.g., every 1000 samples).
2. **Online gradient descent with fairness penalty** (e.g., adding a fairness regularizer at each step).
3. **Static unlearning** (once at deployment, no adaptation).

Metrics:

- **Fairness stability**: Standard deviation of SPD/EOD over time; number of violations.
- **Computational cost**: Total training/unlearning time, memory overhead.
- **Predictive utility**: Area under ROC curve (AUC) or accuracy on a hold-out test stream.
- **Unlearning efficiency**: Time per unlearning request vs. full retraining.

3.10 Step 10: Implementation Details and Algorithms

- **Libraries**: PyTorch or JAX for automatic differentiation; `scikit-learn` for baselines; `influence` package for influence functions.
- **Hessian approximation**: Use Fisher information matrix or conjugate gradient.
- **Data stream**: Use `river` or custom iterator with controlled drift (e.g., rotating hyperplane, changing covariance).

4 Pseudo-Code for the Main Loop

Algorithm 1 Adaptive Fair Unlearning Framework

Require: Initial model θ, training buffer $\mathcal{D}_0$, sliding window size L, threshold τ, unlearning batch size n_u

```
Initialize fairness monitor M, influence estimator I
for each incoming batch B_t do
    ŷ_t ← predict(θ, B_t)
    Update sliding window W ← W ∪ {(ŷ_t, y_t, A_t)}
    f_curr ← compute_fairness(W)
    if CUSUM(f_curr) > τ then
        U ← identify_influential(I, θ, B_t, D_recent)
        θ ← unlearn(θ, U, D_anchor)
        log(timestamp, |U|, f_curr, cost)
    end if
    optionally fine-tune θ on a small replay buffer to preserve utility
    D_recent ← D_recent ∪ B_t (pop oldest if size > N_max)
end for
```

5 Expected Outcomes and Validation

The final deliverable is a Python library implementing the above algorithm. On synthetic and real-world streaming datasets, we expect:

- **Lower fairness violations** (50% reduction compared to periodic retraining).
- **10–100× speedup** over full retraining for each unlearning request.
- **Negligible utility loss** ($< 2\%$ AUC drop) relative to a model trained without unlearning.

The audit trail will also enable post-hoc analysis of bias sources, satisfying accountability requirements.